

Uart_Esp32 - Projektdokumentation - Firmware

Projektname: Uart_Esp32

Dokumentenart: Projektdokumentation

Thema: Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART-Monitor/Decoder mit gezieltem Feld-5-TX-Replay

Erstellungsdatum: 2026-09-17

Autor / verantwortliche Person: OpenAI ChatGPT (technische Erstellung)

Version: v0.11

Status: Entwurf / lokale Build- und Hardwarepruefung ausstehend

Aenderungsdatum: 2026-09-17

Ablageort: Projektordner Uart_ESP32/docs

Referenzen: Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf;
ESP32_UART_Protokoll_Entschluesselung(1).pdf; Projektstand v0.10

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projekt in Uart_Esp32 umbenannt und Dokumentationsregeln umgesetzt
v0.3	2026-09-17	OpenAI ChatGPT	Entwurf	Buildfehler durch typensichere Integer-Vergleiche behoben
v0.4	2026-09-17	OpenAI ChatGPT	Entwurf	Arduino-Secrets, NVS-Fallbacks und WLAN-Provisionierung ergaenzt
v0.5	2026-09-17	OpenAI ChatGPT	Entwurf	WLAN-State-/Fallback-Logik, Recovery-AP, DHCP-Fallback und Resetdiagnose ergaenzt
v0.6	2026-09-17	OpenAI ChatGPT	Entwurf	Compilerfehler in markWifiConnected() behoben
v0.7	2026-09-17	OpenAI ChatGPT	Entwurf	UART Raw/Sniffer, Protokoll-Decoder, Feld-Aliase, Kalibrierung und UART-Webseite ergaenzt
v0.8	2026-09-17	OpenAI ChatGPT	Entwurf	UART-Konfiguration und Live-Monitor getrennt; Raw/Decoder als persistenter Dauerbetrieb dokumentiert
v0.9	2026-09-17	OpenAI ChatGPT	Entwurf	UART-Unterseiten

Version	Datum	Bearbeiter	Status	Aenderung
				eindeutig geroutet; Status-API repariert; Livezaehler und API-Diagnose verbessert
v0.10	2026-09-17	OpenAI ChatGPT	Entwurf	Explizite UART-/Decoder-Laufzeitsteuerung mit START und STOP auf der Monitorseite ergaenzt
v0.11	2026-09-17	OpenAI ChatGPT	Entwurf	Nicht-blockierenden Feld-5-TX-Replay mit -0,5/+0,5 fuer ca. 1 s bei 50 Hz ergaenzt

Ziel des Standes v0.11

Der Projektstand v0.11 ergaenzt den bestehenden UART-Monitor/Decoder um einen bewusst eng begrenzten Sendetest. Der ESP32 kann das **zuletzt frisch empfangene und gueltige Hauptpaket 0x10/0x15** als Vorlage einfrieren, ausschliesslich **Feld 5** auf einen normierten Zielwert von -0,5 oder +0,5 setzen, die XOR-Pruefsumme neu berechnen und diese Paketkopie fuer ungefaehr **1 Sekunde mit 50 Hz** ueber den konfigurierten TX-GPIO senden.

Die Sendeausgabe ist nicht blockierend. UART-RX, Paketparser, Webserver, MQTT und die restliche Firmware koennen waehrend des einsekundigen Replays weiterlaufen. Das zu sendende Paket wird beim Start der Sendung eingefroren; neue RX-Pakete veraendern die laufende TX-Serie daher nicht. Nach UART-START bzw. einer UART-Neukonfiguration muss zuerst wieder ein frisches gueltiges Hauptpaket empfangen werden, bevor TX freigegeben wird.

Der TX-Test ist nur verfuegbar, wenn der Protokoll-Decoder laeuft, ein gueltiger TX-GPIO gesetzt ist und eine frische Hauptpaketvorlage vorhanden ist. Ohne diese Voraussetzungen bleiben die Sendeaktionen gesperrt bzw. liefern eine eindeutige Fehlermeldung.

UART-Betriebsarten

Modus	Verhalten
Aus	Zusaetzliche Hardware-UART ist deaktiviert.
Raw / Sniffer	Bytes werden empfangen und in einem 512-Byte-Ringpuffer gespeichert. Die letzten 256 Bytes werden live als HEX und ASCII angezeigt.
Protokoll-Decoder	Raw-Erfassung bleibt aktiv; zusaetlich werden Pakete synchronisiert, geprueft und die bestaetigten Hauptpakete dekodiert.

UART0 bleibt fuer den seriellen Debug-Monitor reserviert. Fuer die Zusatzfunktion sind Hardware-UART1 und UART2 wahlbar. RX und TX sind frei konfigurierbar; TX kann mit -1 deaktiviert werden. Standard ist RX GPIO16, TX deaktiviert, 115200 Baud, 8N1.

Dauerbetrieb: Die UART-Verarbeitung wird in `loop()` kontinuierlich aufgerufen und besitzt keine Messanzahl oder Laufzeitbegrenzung. Der 512-Byte-Ringpuffer kann dabei nicht volllaufen: Sobald er gefuehlt ist, werden nur die jeweils aeltesten Rohbytes ueberschrieben. Die Dekodierung selbst laeuft weiter. Das Oeffnen oder Schliessen der Weboberflaeche beeinflusst den Decoder nicht.

Nicht zugelassen sind GPIO6..11 wegen des externen Flashs sowie GPIO1/3, damit der Debug-UART nicht kollidiert. GPIO34..39 sind nur als RX geeignet, da sie eingangs-only sind.

Decoder gemaess Messunterlage

Der Decoder verwendet die in ESP32_UART_Protokoll_Entschluesselung(1).pdf dokumentierte Struktur:

1. Suche nach FF.
2. Folgendes Byte muss FB sein.
3. Typ und Laengenbyte lesen.
4. Gesamtlänge als 4 + Länge bestimmen.
5. Vollständiges Paket sammeln.
6. XOR-Prüfsumme ueber Byte 6 bis zum Byte vor der Prüfsumme bilden.
7. Nur gueltige Pakete weiter auswerten.

Fuer das bestaetigte Hauptpaket Typ 0x10, Länge 0x15 ergibt sich eine Gesamtlänge von 25 Bytes. Die sechs 16-Bit-Felder liegen Little Endian ab Byte 12:

Verifikationshinweis: Die Implementierung folgt der im PDF textlich angegebenen XOR-Regel (Byte 6 bis zum Byte vor der Prüfsumme). Die im PDF abgedruckte einzelne Beispiel-Bytefolge ergibt bei einer unabhaengigen Nachrechnung nicht denselben Endwert wie das dort gezeigte letzte Byte. Deshalb zeigt die Firmware gueltige/ungueltige Paketaehler separat an; der Hardwaretest muss bestaetigen, ob die XOR-Grenzen fuer die realen Mitschnitte exakt stimmen. Der Raw-Modus bleibt hiervon unberuehrt.

Technisches Feld	Byteposition	Behandlung
Feld 1	12 / 13	uint16 LE, normiert
Feld 2	14 / 15	uint16 LE, normiert
Feld 3	16 / 17	uint16 LE, normiert
Feld 4	18 / 19	uint16 LE, normiert
Feld 5	20 / 21	uint16 LE, normiert
Feld 6	22 / 23	uint16 LE, nur Rohwert
Checksumme	24	XOR

Normierung und Kalibrierung

Feld 1 bis Feld 5 werden stueckweise linear auf -1.0 ... +1.0 normiert. Minimum, Mitte und Maximum sind fuer jedes Feld separat einstellbar und werden in NVS gespeichert. Als Startwerte werden die Messwerte aus der Referenz verwendet:

Feld	Minimum	Mitte	Maximum
Feld 1	1000	1503	2000
Feld 2	1000	1486	2000
Feld 3	1000	1493	2000
Feld 4	1006	1511	2000
Feld 5	1000	1498	2000

Eine einstellbare Totzone wird nach der Normierung angewendet; Standard ist 0.03. Feld 6 wird bewusst nicht normiert, da die Messunterlage seine Bedeutung noch nicht abschliessend klaert.

Feld-Aliase

Fuer jedes Feld steht ein Textfeld mit maximal 32 Zeichen bereit. Die Darstellung bleibt immer zweistufig:

- feste technische Kennung: Feld 1, Feld 2, ... Feld 6
- optionaler Benutzer-Alias: z. B. Roll, Pitch, Schieber, noch unbekannt

Der Alias ersetzt die technische Kennung nicht. Aliase werden in NVS gespeichert und im Webinterface sowie ueber MQTT ausgegeben.

Feld-5-TX-Replay

Der Sendemodus verwendet **keinen frei erzeugten Paketrahmen**, sondern immer eine Kopie des zuletzt gueltig empfangenen 25-Byte-Hauptpakets. Damit bleiben Header, Status-/Metadatenbytes, Feld 1 bis Feld 4, Feld 6 und alle derzeit semantisch noch unbekannten Bytes unveraendert.

Beim Druucken einer der beiden Schaltflaechen wird folgender Ablauf ausgefuehrt:

1. Pruefen, ob Decoder, TX-GPIO und eine frische Hauptpaketvorlage vorhanden sind.
2. Letztes gueltiges FF FB 10 15 ...-Paket in einen separaten TX-Puffer kopieren.
3. Gewaehlten Zielwert -0,5 oder +0,5 mit der vorhandenen Min/Mitte/Max-Kalibrierung von **Feld 5** in einen 16-Bit-Rohwert zurueckrechnen.
4. Nur Byte 20/21 (Feld 5, Little Endian) ersetzen.
5. XOR-Pruefsumme ueber Byte 6 bis zum Byte vor der Pruefsumme neu bilden und Byte 24 aktualisieren.
6. Das eingefrorene Paket fuer ca. 1000 ms im Raster von 20 ms senden, entsprechend ungefaehr 50 Hz.

Mit den Default-Kalibrierwerten fuer Feld 5 (min=1000, center=1498, max=2000) ergeben sich ungefaehr:

Ziel normiert	Rohwert Feld 5	Berechnung
-0,5	1249	$1498 - 0,5 * (1498 - 1000)$
+0,5	1749	$1498 + 0,5 * (2000 - 1498)$

Die optionale Totzone wird beim Zurueckrechnen **nicht** invertiert; der Sendewert bezieht sich direkt auf die lineare Min/Mitte/Max-Normierung gemaess Messunterlage.

Die Implementierung verwendet einen nicht-blockierenden TX-Zustandsautomaten in `UartManager::loop()`. Pro Loop-Durchlauf wird hoechstens ein faelliges Paket geschrieben. Bei groesserer Laufzeitverzoegerung werden keine langen Paketbursts nachgeholt.

Sicherheits-/Hardwarevoraussetzung: TX darf erst elektrisch verbunden werden, wenn Pegel, Richtung und Bus-Topologie geklaert sind. ESP32-GPIOs arbeiten mit 3,3 V und sind nicht 5-V-tolerant.

Webinterface

Ab v0.11 sind Konfiguration und Beobachtung bewusst getrennt. Dadurch bleibt die Live-Seite uebersichtlich, waehrend die umfangreichen Einstellungen auf einer eigenen Seite liegen.

UART Monitor /uart/monitor

Die Monitorseite enthaelt Laufzeitinformationen und eine direkte Laufzeitsteuerung:

- **START** zum erneuten Initialisieren und Starten des gespeicherten UART-/Decoder-Modus,
- **STOP** zum sofortigen Anhalten des aktuellen UART-Empfangs/Decoders, ohne die NVS-Konfiguration zu aendern,
- aktuellen UART-/Decoder-Status,
- fortlaufenden Bytezaehler,
- Feld 1 bis Feld 6 mit technischer Kennung, Alias, Rohwert und - fuer Feld 1 bis 5 - normiertem Wert,
- Paketzaehler fuer gesamt, gueltig, ungueltig und 0x10/0x15,
- die letzten 256 Rohbytes als HEX und ASCII,
- Funktion zum Leeren des Raw-Ringpuffers und Bytezaehlers,
- direkten Link zur UART-Einstellungsseite,

- eigenen TX-Bereich fuer Feld 5 = -0,5 und Feld 5 = +0,5, jeweils ca. 1 s / 50 Hz,
- sichtbaren TX-Status, Zielwert, berechneten Feld-5-Rohwert und Zahl der bereits gesendeten Pakete.

Die Livewerte werden ueber /uart/status als JSON viermal pro Sekunde (ca. 250 ms) aktualisiert. Die Seite prueft zusaetzlich, ob wirklich application/json geliefert wird und zeigt den Status der API sichtbar an. Die initialen Byte-/Paketwerte werden bereits serverseitig mit dem aktuellen Stand befuellt, sodass die Anzeige auch vor dem ersten JavaScript-Refresh konsistent ist. Das JSON-Feld **running** steuert ausserdem den Zustand der START-/STOP-Schaltflaechen: START ist bei laufendem UART deaktiviert, STOP bei angehaltenem UART.

Ohne manuellen Eingriff laeuft die Dekodierung weiterhin dauerhaft im Hintergrund. Ein manueller STOP pausiert nur die aktuelle Laufzeitinstanz; die NVS-Konfiguration bleibt bestehen.

UART Einstellungen /uart/settings

Die separate Einstellungsseite enthaelt:

- Modus Aus, Raw / Sniffer (Dauerbetrieb) oder Protokoll-Decoder (Dauerbetrieb),
- Auswahl Hardware-UART1 oder UART2,
- RX- und optionalen TX-GPIO,
- Baudrate und Format 8N1, 8E1, 8O1, 8N2,
- Alias-Textfelder fuer Feld 1 bis Feld 6,
- Min/Mitte/Max-Kalibrierung fuer Feld 1 bis Feld 5,
- einstellbare Totzone,
- Anzeige des aktuellen gespeicherten Modus und UART-Status.

Beim Speichern werden die Werte in NVS abgelegt und nur die UART-Schnittstelle neu initialisiert. Ein gespeicherter Raw- oder Decoder-Modus wird beim naechsten ESP32-Start automatisch wieder aktiviert.

Web-Routen

Route	Methode	Funktion
/uart/status	GET	JSON-Livestatus; wird zuerst registriert, damit keine kuerzere UART-Route die API abfaengt
/uart/settings	GET	Explizite UART-Konfiguration, Feld-Aliase und Kalibrierung
/uart/monitor	GET	UART-Liveansicht / Decoder-Monitor mit START-/STOP-Steuerung
/uart/start	POST	Startet den gespeicherten Raw-/Decoder-Modus mit der aktuellen UART-Konfiguration; NVS bleibt unveraendert
/uart/stop	POST	Stoppt Hardware-UART und Decoder zur Laufzeit; NVS bleibt unveraendert
/uart/send_f5_neg	POST	Startet Paketkopie-Replay mit Feld 5 = -0,5 fuer ca. 1 s bei 50 Hz
/uart/send_f5_pos	POST	Startet Paketkopie-Replay mit Feld 5 = +0,5 fuer ca. 1 s bei 50 Hz
/uart	GET	Kompatibler Redirect auf /uart/monitor; wird als letzte UART-GET-Route registriert
/save_uart	POST	Einstellungen, Aliase und Kalibrierung in NVS speichern; UART neu initialisieren
/uart_clear	POST	Raw-Ringpuffer und Bytezaehler leeren; Decoderbetrieb bleibt aktiv

START-/STOP-Verhalten

Die Laufzeitsteuerung trennt bewusst **Konfiguration** und **aktuellen Betriebszustand**:

Aktion	Wirkung	Persistenz
START	Initialisiert den konfigurierten Hardware-UART, setzt den Paketparser auf Synchronisationssuche und startet Raw-/Decoder-Verarbeitung	keine Aenderung an NVS
STOP	Beendet den Hardware-UART, stoppt Raw-/Decoder-Verarbeitung und setzt den Paketparser zurueck	keine Aenderung an NVS
Neustart ESP32	Liest den gespeicherten Modus aus NVS und startet Raw/Decoder automatisch, sofern der Modus nicht Aus ist	gespeicherter Modus bleibt massgeblich
Modus Aus	START ist nicht sinnvoll und wird auf der Monitorseite deaktiviert	persistent ueber UART-Einstellungen

STOP loescht bewusst **nicht** den Raw-Ringpuffer, die Byte-/Paketaehler oder die zuletzt dekodierten Werte. Damit kann nach einer Messpause der letzte Zustand weiterhin untersucht werden. Zum gezielten Zuruecksetzen steht die vorhandene Funktion Rohdatenpuffer / Bytezaehler leeren separat zur Verfuegung.

Ursache des beobachteten Fehlers und Korrektur

Die Screenshots des Hardwaretests zeigen zwei Symptome, die dieselbe Ursache haben:

1. Aufruf von /uart/settings, aber Anzeige der Monitorseite.
2. Raw-Daten sind sichtbar, waehrend Byte-/Paketaehler auf 0 bleiben und keine dekodierten Felder erscheinen.

Die Raw-Daten wurden serverseitig direkt aus dem Ringpuffer in die HTML-Seite geschrieben. Die Zaehler und Decoderfelder sollten dagegen anschliessend ueber JavaScript von /uart/status aktualisiert werden. Da die kurze Route /uart die Unterroute /uart/status abfing, erhielt fetch() eine HTML-Seite statt JSON. Der bisherige JavaScript-Fehlerpfad unterdrueckte die Ausnahme; deshalb blieb die Anzeige bei den statisch vorgegebenen Nullwerten.

Die in v0.9 eingefuehrte Routing-Korrektur besteht aus vier Massnahmen:

- spezifische /uart/*-Routen vor /uart registrieren,
- eigene Live-Route /uart/monitor verwenden,
- /uart nur als spaeten Redirect beibehalten,
- Live-JavaScript um sichtbare API-/Content-Type-Diagnose erweitern.

Da in den Raw-Daten bereits wiederholt FF FB 10 15 sichtbar ist, sollte der Decoder nach funktionierender Status-API Paketaehler und Felddaten liefern. Die implementierte XOR-Regel passt zu den im Screenshot sichtbaren Beispielpaketen; der weitere Hardwaretest bleibt dennoch massgeblich.

MQTT-Erweiterung

Der normale Heartbeat veroeffentlicht zusaetzlich den UART-Modus und den empfangenen Bytezaehler. Unter <Basis>/uart/ werden ausgegeben:

- mode

- status
- bytes
- packets
- valid_packets
- invalid_packets
- field1/raw bis field6/raw, sobald ein Hauptpaket dekodiert wurde
- field1/norm bis field5/norm
- field1/alias bis field6/alias retained

Der Raw-HEX-Datenstrom wird bewusst nicht vollstaendig ueber MQTT gespiegelt, um Broker und Funkverbindung nicht unnoetig zu belasten.

Persistenz / NVS

UART-Einstellungen werden im separaten Namespace `uartconf` gespeichert. Gespeichert werden Betriebsart, Hardware-UART, RX/TX, Baudrate, Frame, Totzone, sechs Aliase und die Kalibrierwerte fuer Feld 1 bis Feld 5. Fehlende Keys werden ohne NVS-Fehlermeldungen mit sicheren Defaults belegt.

Elektrische Voraussetzungen

Der ESP32 arbeitet mit 3,3-V-Logik und seine GPIOs sind nicht 5-V-tolerant. Vor dem Anschluss gilt:

- gemeinsame Masse zwischen ESP32 und Zielgeraet herstellen,
- zuerst nur RX anschliessen und passiv sniffen,
- High-Pegel der UART-Leitung mit Multimeter/Oszilloskop pruefen,
- bei 5-V-UART Pegelwandler oder geeigneten Spannungsteiler verwenden,
- TX erst nach geklaerter Richtung und Bus-Topologie verbinden.

Projektstruktur v0.11

```
Uart_ESP32/
.gitignore
platformio.ini
README.md
include/
  arduino_secrets.example.h
  arduino_secrets.h      # lokal, nicht in Git / nicht im Auslieferungs-ZIP
src/
  ConfigDefaults.h
  BatteryMonitor.cpp/.h
  DeepSleepManager.cpp/.h
  MqttManager.cpp/.h
  OtaManager.cpp/.h
  UartManager.cpp/.h     # neu v0.7
  main.cpp
docs/
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.11.*
  2026-09-17 - Uart_Esp32 - Commit-Nachricht - Feld5-TX-Replay - v0.11.txt
```

Hinweis: Der Projektname bleibt Uart_Esp32. Der Root-Ordner Uart_ESP32 wird fuer diese Auslieferung unveraendert aus dem vom Nutzer bereitgestellten Paket uebernommen, entsprechend der Regel zur Beibehaltung des Codepaket-Root-Ordners.

Geaenderte / neue Dateien in v0.11

- README.md
- src/ConfigDefaults.h
- src/UartManager.h
- src/UartManager.cpp
- src/main.cpp
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.11.md (neu)
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.11.docx (neu)
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.11.pdf (neu)
- docs/2026-09-17 - Uart_Esp32 - Commit-Nachricht - Feld5-TX-Replay - v0.11.txt (neu)

Pruefung und offene Verifikation

Die Aenderungen wurden statisch auf Parsergrenzen, Paketkopie, Feld-5-Byteposition, Little-Endian-Schreibweise, XOR-Neuberechnung, nicht-blockierendes 20-ms-Senderaster, TX-Voraussetzungen und Web-Routen geprueft. Ein echter PlatformIO-Build mit dem ESP32-Arduino-Framework ist in der Erstellungsumgebung weiterhin nicht verfuegbar. Verbindlicher lokaler Test:

```
pio run -e uart_esp32_usb -t clean
pio run -e uart_esp32_usb
pio run -e uart_esp32_usb -t upload
```

Hardwaretest in dieser Reihenfolge:

1. Protokoll-Decoder mit RX-only starten und bestaetigen, dass gueltige 0x10/0x15-Pakete dekodiert werden.
2. Pegel, Richtung und Bus-Topologie elektrisch pruefen.
3. Einen geeigneten TX-GPIO in /uart/settings konfigurieren und TX erst danach verbinden.
4. Nach UART-START warten, bis im TX-Bereich bereit: gueltige Paketvorlage vorhanden angezeigt wird.
5. Feld 5 = -0,5 fuer ca. 1 s senden ausloesen. Erwartet sind ungefaehr 50 Pakete und mit Defaultkalibrierung ein Feld-5-Rohwert um 1249.
6. Danach Feld 5 = +0,5 fuer ca. 1 s senden pruefen. Erwartet sind ungefaehr 50 Pakete und ein Rohwert um 1749.
7. Parallel kontrollieren, dass RX-/Decoderzaehler weiterlaufen und keine Software-Resets auftreten.

Die reale elektrische und funktionale Wirkung am Zielgeraet muss vor einem produktiven Einsatz separat verifiziert werden.

Auslieferungsform

Bereitgestellt werden ein vollstaendiges ZIP mit unveraendertem Root-Ordner Uart_ESP32, ein ZIP mit nur geaenderten/neuen Dateien in derselben Struktur, eine Patch-Datei, eine englische Commit-Nachricht mit Datum und vollstaendiger Dateiliste sowie die Projektdokumentation als PDF und bearbeitbare DOCX mit identischem Stand v0.11.